

GWBP: Accelerating Weighted Back-Projection for Image Reconstruction via Efficient GPU Parallel Optimization

Hongbo Miao¹ and Haodong Bian^{1,2} ✉[0000–0003–0907–288X]

¹ School of Computer Technology and Applications, Qinghai University, Xining
810016, China

² Intelligent Computing and Applications Laboratory of Qinghai Province, Qinghai
University, Xining 810016, China

Abstract. Weighted Back-Projection (WBP) is a classical and widely used reconstruction method in tomography and cryo-electron tomography (cryo-ET). Given a tilt series acquired at different angles, WBP forms a 3D volume by applying weighting and accumulating geometry-dependent back-projection contributions across all tilts, making it attractive for its simplicity and robustness. However, modern cryo-ET datasets continue to grow in tilt count, volume size, and resolution, and often incorporate costly three-dimensional CTF correction (3D-CTF correction). As a result, WBP increasingly dominates end-to-end runtime, while conventional CPU pipelines suffer from insufficient throughput and limited scalability. **We propose GWBP, a highly optimized GPU implementation of the WBP parallel algorithm.** First, GWBP performs kernelization of per-tilt operators and maintains persistent residency of intermediate data on the GPU. This design bypasses redundant correction steps on the host and minimizes host-device communication. Next, to enhance computational efficiency, we propose fragmentation-aware task batching and temporal buffer reuse. This synergistic approach effectively minimizes memory access. Finally, a stream-oriented scheduling scheme is introduced to overlap communication with computation, thereby eliminating pipeline bubbles and enhancing overall throughput. Compared to the state-of-the-art multi-core CPU implementation, GWBP reduces end-to-end runtime from 3.131 s to 0.655 s, achieving up to a $4.78\times$ speedup.

Keywords: Weighted Back-Projection · GPU acceleration · Cryo-ET · 3D-CTF · Pipeline optimization.

1 Introduction

Cryo-electron tomography (cryo-ET) has become an important tool for visualizing macromolecular structures by reconstructing three-dimensional volumes from

✉ Corresponding author: hpc_bhd@163.com

a tilt series acquired at multiple viewing angles[15]. At the core of this modality is Weighted Back-Projection (WBP), a classical direct reconstruction method that forms a 3D volume by weighting and back-projecting the tilt images [13]. Due to its simplicity, robustness, and compatibility with established processing workflows, WBP remains a common choice in electron tomography and cryo-ET pipelines, where it is often coupled with weighting and contrast-transfer-function (CTF) related corrections [16].

Historically, efforts have focused on the theoretical refinement of WBP to reconcile its performance with Fourier-based methods [14]. Subsequent innovations have sought to optimize the projection operators themselves; for instance, Zhou et al. leveraged cross-correlation-based back-projection to suppress artifacts in GPR imaging [20], while Long et al., in X-ray CT reconstruction, proposed separable footprint (SF) forward and back-projection methods to improve the accuracy and efficiency of 3D cone-beam projection, which improves reconstruction accuracy for large-cone-angle rays, but also introduces higher computational cost due to the more complex trapezoidal approximation in the axial direction[10].

Challenges. While significant efforts have been dedicated to algorithmic refinement, the burgeoning scale of dataset resolution, tilt count, and reconstruction volume has increasingly exacerbated computational overhead. Our analysis identifies CPU-based implementations of back-projection as a critical limiting factor in overall throughput. This inefficiency is twofold: first, the repetitive accumulation of voxel values across numerous projections creates redundant write operations; second, the geometry-driven interpolation induces irregular memory access, which collectively strains the cache hierarchy and impede memory throughput [19]. Furthermore, end-to-end performance is frequently bottlenecked by pipeline inefficiencies, including the complex orchestration of weighting, 3D-CTF correction, and accumulation, alongside the latency penalties of redundant resource allocation and high-volume I/O for MRC output [4]. The computational burden is further amplified when 3D-CTF correction is enabled, as the requirement to process multiple depth-dependent layers for each tilt significantly scales the total runtime [9].

Key Insights. In response to these challenges, we propose GWBP, a highly-optimized GPU implementation of the WBP parallel algorithm. GWBP overcomes CPU-side throughput limitations by mapping the WBP algorithm onto the high-density compute resources of the GPU, utilizing an optimized streaming execution model to saturate device bandwidth. Its primary advantages are three-fold: ❶ it kernelizes each skewed operator and maintains data residency on the device to minimize host-to-device overhead; ❷ it introduces a fragmentation-aware task batching strategy and a temporal buffer reuse mechanism to effectively reduce memory access latency and overhead; and ❸ by strategically scheduling kernel execution through stream operations, the system overlaps communication with computation (including transmission, rectification, and accumulation), thereby minimizing pipeline bubbles and maximizing overall throughput.

Contributions. The main contributions of this work are as follows:

- **Persistent Residency.** At the heart of our approach is the kernelization of per-tilt operators and the persistent residency of intermediate data on the GPU; this design bypasses redundant correction steps on the host and minimizes host-device communication.
- **Task Batching and Resource Reuse.** To enhance computational efficiency, we propose fragmentation-aware task batching and temporal buffer reuse. This synergistic approach effectively minimizes memory access.
- **Efficient Stream-Based Scheduling.** A stream-oriented scheduling scheme is introduced to overlap communication with computation (transfer, correction, and accumulation), thereby eliminating pipeline bubbles and enhancing overall throughput.

2 Background and Related Work

2.1 Weighted Back-Projection with 3D-CTF

Given a tilt series $\{I_n\}_{n=1}^N$ with angles $\{\theta_n\}$ and per-tilt weights $\{w_n\}$, the baseline computes, for each voxel (x, z, y) , a detector coordinate u_n via a rigid rotation around (x_0, z_0) :

$$u_n(x, z) = (x - x_0) \cos \theta_n - (z - z_0) \sin \theta_n + x_0. \quad (1)$$

The reconstruction accumulates contributions from all tilts:

$$V(x, z, y) = \sum_{n=1}^N w_n \cdot \hat{I}_n(u_n(x, z), y), \quad (2)$$

where $\hat{I}_n$ denotes the tilt image after weighting and 3D-CTF correction. Sampling on non-integer u follows the baseline linear interpolation:

$$\hat{I}_n(u, y) = (1 - \alpha) \hat{I}_n(t, y) + \alpha \hat{I}_n(t + 1, y), \quad t = \lfloor u \rfloor, \quad \alpha = u - t. \quad (3)$$

When 3D-CTF correction is enabled, the baseline applies a depth-sliced correction. For each voxel location and tilt, a slice index is selected by discretizing the tilt-dependent depth coordinate with step Δz and an offset h :

$$n_z = \left\lfloor \frac{z_n(x, z) + h}{\Delta z} \right\rfloor. \quad (4)$$

2.2 Related Work

Algorithmic and CPU-level Optimizations. A substantial body of work has focused on improving the performance of WBP-based reconstruction at the algorithmic and CPU-system levels. Widely used software such as IMOD provides an end-to-end tomography workflow [8]. On multi-core CPUs, Tomo3D and its subsequent versions accelerate reconstruction through parallelization and low-level optimizations, including vectorization and careful handling of disk access [1, 2]. These efforts demonstrate that even for classical algorithms, production-scale datasets and integrated pipelines demand coordinated optimization across computation mapping, data organization, synchronization, and write-back.

GPU-accelerated Optimizations. GPU acceleration has been widely explored for tomography and reconstruction. Toolboxes such as ASTRA provide GPU implementations of projection and back-projection operators and support a range of reconstruction algorithms [12]. AreTomo integrates marker-free alignment and reconstruction in a GPU-accelerated workflow [18]; Tomo Live enables on-the-fly alignment and reconstruction during acquisition [5]; and RELION-5 extends electron tomography processing within a standardized framework [3]. From a performance-engineering perspective, TiltRec further highlights renewed interest in high-throughput classical reconstruction on both CPUs and GPUs [7]. Overall, these efforts suggest that practical performance depends increasingly on end-to-end coordination across correction, reconstruction, and I/O, rather than on a single accelerated kernel alone.

3 Motivation

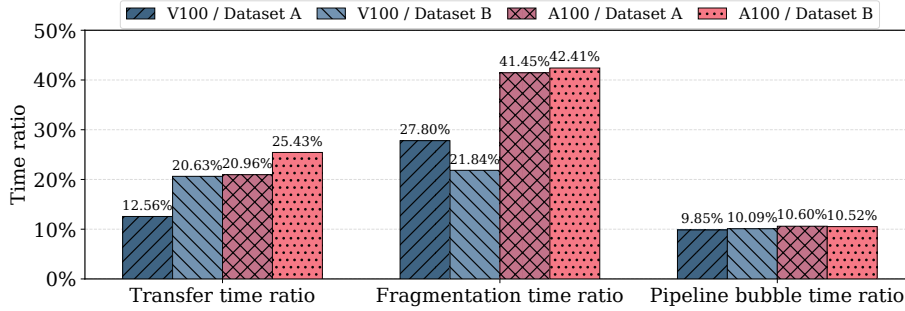


Fig. 1. WBP algorithm hotspot overhead analysis on NVIDIA V100 and A100.

To deeply understand the performance landscape of the WBP algorithm, we conduct a comprehensive profiling across diverse hardware architectures (NVIDIA V100 and A100 GPUs) and representative datasets. As illustrated in Figure 1, we identify three critical bottlenecks that impede the execution efficiency of the WBP algorithm:

- **Excessive Host to Device Data Transfer.** Our breakdown analysis shows that data movement between the Host and Device consumes 12.56% to 25.43% of the total execution time. This overhead stems from the high-frequency synchronization necessitated by WBP computational characteristics. (Motivation 1)
- **Task Fragmentation.** The most prominent bottleneck lies in fragmented task execution, accounting for 21.84% to 42.41% of the runtime. This phenomenon is characterized by a massive number of small-scale kernels processing granular data. (Motivation 2)

- **Pipeline Bubbles.** We observe persistent pipeline bubbles during sequential kernel execution, which stagnate the GPU for 9.85% to 10.52% of the total wall-clock time. (Motivation 3)

Motivation 1

The excessive host-device interaction indicates that current methods fail to maintain intermediate data persistence on the device, thereby failing to mitigate the significant latency incurred by frequent data transfers.

Motivation 2

Such fragmentation results in diminished computational intensity and suboptimal data reusability, highlighting the need for fragmentation-aware task batching and resource recycling to bolster execution efficiency.

Motivation 3

Rigid dependency chains that hinder overlapping computation and memory access. To reclaim these lost cycles, it is imperative to achieve asynchronous execution by leveraging multi-stream concurrency.

4 Method

4.1 Overview

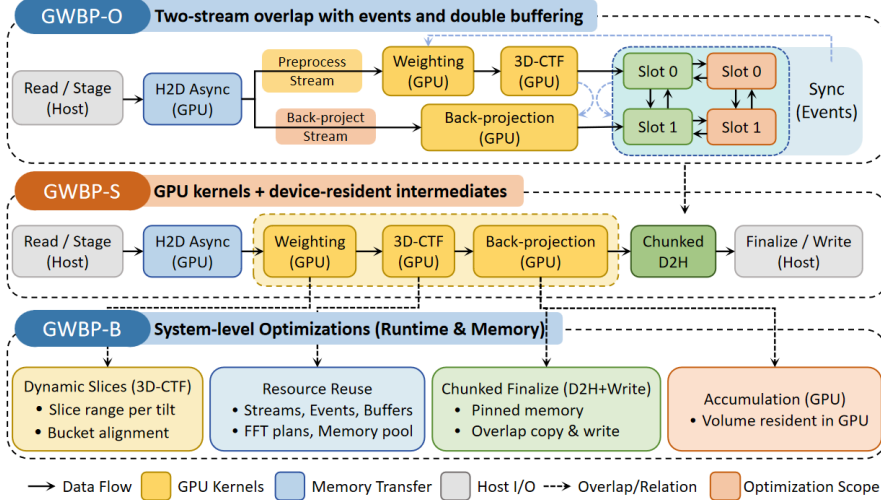


Fig. 2. End-to-end workflow and where each GWBP stage applies.

GWBP keeps the same geometric mapping, interpolation rule, weighting operator, and 3D-CTF correction model as the baseline. Figure 2 summarizes

the end-to-end workflow and shows where the three cumulative optimization stages apply: GWBP-S for GPU kernelization with persistent residency, GWBP-B for task batching and resource reuse, and GWBP-O for efficient stream-based scheduling. The core principle is to optimize the *pipeline* rather than a single kernel: keep intermediate data on device, avoid unnecessary 3D-CTF work, reduce host coordination and write-back stalls, and overlap preprocessing with back-projection.

This definition is preserved in GWBP; our optimizations only change how efficiently the corresponding corrected slices are produced, transferred, and consumed.

4.2 GWBP-S: GPU Kernelization with Persistent Residency

GWBP-S targets the dominant per-tilt computation path. Its core idea is to move weighting, 3D-CTF correction, and back-projection to GPU kernels and connect them through persistent device buffers, so that large intermediates no longer need to be materialized on the host between stages. In this way, GWBP-S bypasses redundant host-side correction steps and minimizes host-device communication along the critical path.

GPU Weighting. The first optimization is to offload the baseline 1D weighting step to the GPU while preserving its original filtering rule. For each tilt, the input micrograph is copied to a device buffer, and weighting is applied there by batched row-wise cuFFT. Each row is linearly padded to the enlarged FFT length, transformed by a batched R2C FFT, multiplied by the same radial/Gaussian weighting rule as in the CPU baseline, and transformed back in place. This design preserves the original weighting behavior, exposes row-level parallelism, and removes host-side FFT processing from the per-tilt critical path.

GPU 3D-CTF Correction with Device-Resident Corrected Stacks. The second optimization is to keep the full 3D-CTF stage on the GPU and avoid host-side staging of corrected slices. Instead of recomputing a full forward FFT for every corrected slice, GWBP-S performs one forward 2D FFT for the current tilt, expands the base spectrum across all required depth offsets on the GPU, and reconstructs the corrected stack by a single batched inverse cuFFT. The slice-dependent phase-flip rule remains identical to the baseline, but the resulting corrected stack is stored persistently in device memory and consumed immediately by the next stage. In this way, GWBP-S bypasses redundant correction steps and removes intermediate device-to-host transfers from the weighting-CTF-back-projection chain.

GPU Back-Projection with Per-tilt Lookup Reuse. The third optimization is to simplify the innermost accumulation path. For a fixed tilt, the mapping from reconstruction coordinates (i, k) in the xz plane to detector coordinates and

slice indices is independent of y . GWBP-S therefore precomputes, once per tilt, an xz lookup map containing the left interpolation index, interpolation coefficient, and slice index. During accumulation, one thread is assigned to one output voxel in the flattened volume layout. Each thread reuses the precomputed mapping, reads two neighboring samples from the corrected stack, performs the same linear interpolation as the baseline, and accumulates directly into the persistent device volume. This removes repeated coordinate transforms and index evaluation from the innermost loop while preserving the numerical semantics of the original formulation.

Overall, GWBP-S converts the original host-coordinated WBP pipeline into a device-resident execution path. The main benefit is not only kernel acceleration itself, but also the elimination of unnecessary intermediate transfers between weighting, 3D-CTF correction, and back-projection, thereby minimizing host-device communication.

4.3 GWBP-B: Task Batching and Resource Reuse

After kernelization, end-to-end runtime is no longer dominated solely by arithmetic throughput. In practice, the remaining bottlenecks come from three sources: unnecessary 3D-CTF work, repeated per-tilt resource setup, and output-stage finalize/write-back overhead. GWBP-B addresses these costs through fragmentation-aware task batching and temporal resource reuse at the system level, thereby reducing unnecessary memory access and coordination overhead.

Fragmentation-Aware Task Batching. The first optimization is a fragmentation-aware batching strategy that avoids correcting slices never referenced by the current tilt geometry. Correcting the full depth range for every tilt is wasteful because only a subset of slices is actually used by Eq. (4). For each tilt n , GWBP-B computes a conservative valid slice interval over the reconstruction domain and corrects only that range:

$$k \in [k_{\min}^{(n)}, k_{\max}^{(n)}], \quad k_{\min}^{(n)} = \left\lceil \frac{z_{\min}^{(n)} - z_0}{\Delta z} \right\rceil, \quad k_{\max}^{(n)} = \left\lceil \frac{z_{\max}^{(n)} - z_0}{\Delta z} \right\rceil, \quad (5)$$

where $z_{\min}^{(n)}$ and $z_{\max}^{(n)}$ are the minimum and maximum transformed depth coordinates attained by all voxels under tilt n .

To preserve baseline indexing, GWBP-B shifts the slice lookup into the reduced local stack. Let n_z^{base} denote the baseline slice index from Eq. (4). The local index is

$$n_z^{\text{local}} = n_z^{\text{base}} - k_{\min}^{(n)}, \quad (6)$$

so that the compact range $\{0, \dots, k_{\max}^{(n)} - k_{\min}^{(n)}\}$ remains numerically equivalent to the original discretization over the valid window.

To avoid highly irregular small FFT batches, we optionally align the slice count to a bucket size B :

$$L^{(n)} = k_{\max}^{(n)} - k_{\min}^{(n)} + 1, \quad \tilde{L}^{(n)} = \left\lceil \frac{L^{(n)}}{B} \right\rceil \cdot B. \quad (7)$$

When needed, the corrected range is conservatively extended to length $\tilde{L}^{(n)}$. This improves batched FFT regularity, stabilizes performance across tilts with different valid slice counts, and avoids unnecessary memory access beyond the valid slice range.

Resource Reuse Across the Tilt Loop. The second optimization is to eliminate repeated setup costs that remain after kernelization. GWBP-B therefore reuses long-lived GPU resources across the full tilt loop. In our implementation, the input image buffer, corrected-stack buffer, depth-offset array, and persistent reconstruction volume are allocated once and reused for all tilts. In the 3D-CTF stage, geometry-dependent resources for a fixed image size are also reused across tilts, while batch-dependent buffers and inverse cuFFT plans are expanded lazily and cached by batch size. This reduces repeated device allocation, plan creation, and auxiliary preprocessing overhead when consecutive tilts share the same geometry, while also minimizing memory access effectively.

Output-Stage I/O Optimization. The third optimization is a batched output strategy that reduces finalize/write-back stalls. Once the reconstruction size is known, GWBP-B creates the output MRC container and preallocates the full file extent before bulk data are written:

$$S_{\text{file}} = 1024 + S_{\text{sym}} + N_x \cdot N_y \cdot N_z \cdot \text{sizeof}(\text{float}), \quad (8)$$

where 1024 is the MRC header size and S_{sym} is the symmetry-data region. GWBP-B then attaches a large fully buffered I/O stream to the output file, so that the final volume write is serviced by fewer, larger host-side flushes rather than many small writes. In practice, this reduces file-growth and metadata-management overhead during finalization and improves sustained output throughput for large reconstructed volumes.

For output compatibility, GWBP-B also computes reconstruction statistics directly from the device-resident volume:

$$v_{\min} = \min_{p \in \Omega} V(p), \quad v_{\max} = \max_{p \in \Omega} V(p), \quad \mu = \frac{1}{|\Omega|} \sum_{p \in \Omega} V(p), \quad (8)$$

and writes them into the MRC header without an additional full-volume host scan. Together, these optimizations reduce per-tilt coordination overhead, unnecessary memory access, and output-stage waiting in the end-to-end pipeline.

4.4 GWBP-O: Efficient Stream-Based Scheduling

After GWBP-B reduces unnecessary work and setup overhead, the main remaining inefficiency is the pipeline bubble between preprocessing and accumulation. In a serialized schedule, weighting and 3D-CTF correction for tilt n must complete before its back-projection can begin, and preprocessing for tilt $n + 1$ is delayed by the accumulation of tilt n . GWBP-O addresses this bottleneck by overlapping the two phases.

Two-Stream Overlapped Schedule. The primary optimization in GWBP-O is to decouple preprocessing from accumulation with two non-blocking CUDA streams, as illustrated in Figure 3. A *preprocess stream* is used for host-to-device transfer, optional weighting, and 3D-CTF correction, while a *back-projection stream* is used for accumulation into the persistent reconstruction volume V . After warm-up, preprocessing of tilt $n + 1$ can proceed concurrently with back-projection of tilt n , thereby overlapping communication with computation and reducing idle time between stages.

Double-Buffered Device Slots. To enable overlap safely, GWBP-O uses two device slots indexed by

$$s(n) = n \bmod 2.$$

Each slot $s \in \{0, 1\}$ contains a pinned host input buffer H_s , a device image buffer D_s^{img} , and a corrected-stack buffer D_s^{ctf} . Tilt n is therefore preprocessed into slot $s(n)$, while the back-projection stream may simultaneously consume the corrected stack from the other slot. This double-buffered organization allows the two streams to overlap without overwriting in-flight data.

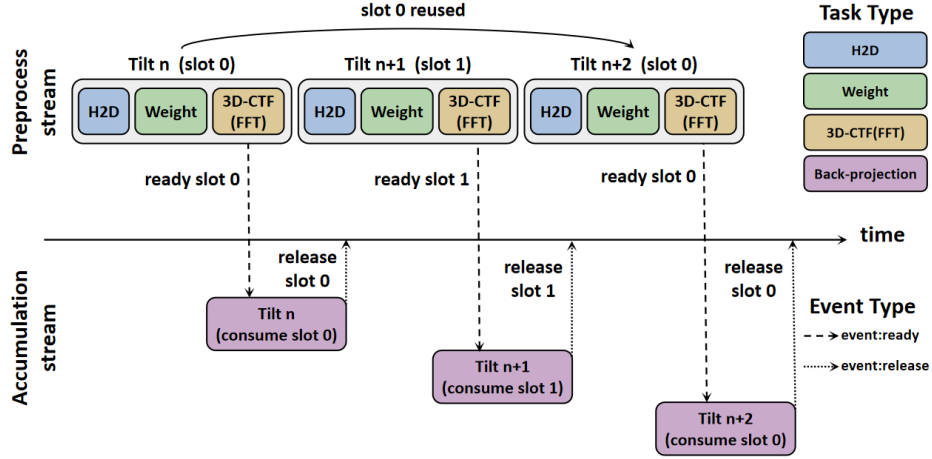


Fig. 3. Two-stream overlap between preprocessing and back-projection in GWBP-O.

Correctness-Preserving Event Synchronization. GWBP-O changes the inter-tilt execution schedule, but not the numerical semantics of WBP. Correctness is preserved by event-based synchronization. After preprocessing finishes for a slot, a ready event is recorded; the back-projection stream waits on this event before consuming the corrected stack. After accumulation completes, a release event is recorded; the preprocess stream waits on this event before reusing the slot. Since all updates to V are issued on the same back-projection stream, accumulation remains serialized and race-free. Thus, GWBP-O improves throughput by removing scheduling bubbles, while keeping the reconstruction behavior equivalent to the baseline.

5 Evaluation

5.1 Experimental Setup

Platforms. We conduct experiments on two systems. Platform 1 runs Ubuntu 22.04.5 on a dual-socket Intel Xeon Gold 6542Y CPU with 48 physical cores (96 threads) and four NVIDIA Tesla V100-PCIE-32GB GPUs. Platform 2 runs Ubuntu 24.04.3 on a dual-socket Intel Xeon Gold 6530 CPU with 64 physical cores (128 threads) and two NVIDIA A100-SXM4-40GB GPUs. Although both platforms contain multiple GPUs, all reported GPU experiments use a single GPU per run. The CPU baseline is derived from prior work on parallel WBP on multi-CPU systems [17], uses OpenMP parallelism, and is executed without explicit CPU affinity restrictions. The software environment includes GCC 10.5.0, Intel ICC 2021.1 Beta, Intel MKL 2021.0 Update 1, and CUDA Toolkit 12.6.

Baseline. The CPU baseline runtimes differ across platforms because the two systems vary in CPU microarchitecture, memory/NUMA behavior, and I/O stack, all of which can noticeably affect a bandwidth- and I/O-sensitive WBP pipeline. In contrast, GWBP reduces sensitivity to host-side variability by minimizing repeated resource creation, staging output to fast local storage when enabled, and performing chunked write-back with overlapped execution. As a result, while absolute runtimes may change between platforms, the relative GPU speedup trend remains consistent.

Datasets. We evaluate on two proteasome datasets, which were obtained from the Electron Microscopy Public Image Archive (EMPIAR) [6]. All runs use identical reconstruction parameters, so that runtime comparisons reflect implementation differences rather than parameter changes.

Validation. We compute the Relative Mean Error (RME) reported by the baseline validation script, and deem a run correct if $RME < 0.001$ under identical reconstruction parameters. Because floating-point operations are not strictly associative and shared clusters exhibit performance jitter, repeated runs may yield slightly different runtimes, whereas the numerical discrepancies remain within the tolerance specified above.

5.2 Results

We compare the CPU baseline with three cumulative GPU variants that represent successive optimization stages in Section 4: GWBP-S, GWBP-B, and GWBP-O. Our results are organized in two parts. We first present a bottleneck breakdown for the CPU baseline and the best (GWBP-O) to identify where time is spent and how each optimization takes effect. We then report the speedup progression and summarize runtime and speedup across platforms and datasets.

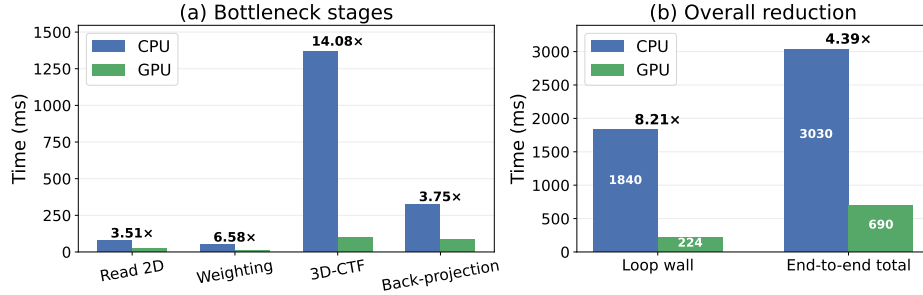


Fig. 4. Stage-level time breakdown comparing the CPU baseline and the best GPU pipeline on the NVIDIA A100 platform with Dataset A.

Bottleneck Breakdown. To attribute performance gains to concrete pipeline components, we instrument both the CPU baseline and GWBP-O with lightweight timing around the major pipeline components. In Figure 4, we report the stage-level costs for CPU-side read/staging (*Read 2D*), weighting, 3D-CTF correction, and back-projection, together with the loop wall time and the end-to-end total runtime. It shows that the CPU baseline is dominated by 3D-CTF correction (1368 ms), followed by back-projection (324 ms), so the loop wall time is largely determined by these compute-heavy stages. After optimization, 3D-CTF achieves the largest reduction at 14.08 $\times$, while weighting and back-projection are reduced by 6.58 $\times$ and 3.75 $\times$, respectively. As a result, the loop wall time drops by 8.21 $\times$ and end-to-end gain remains substantial at 4.39 $\times$, showing that GPU kernelization, device-resident execution, and reduced in-loop coordination are effective in removing the critical bottlenecks.

Ablation Study. Guided by the bottleneck analysis, we next evaluate how performance improves when optimizations are introduced in a cumulative manner. Figure 5 summarizes the progression from the CPU baseline to GWBP-S, GWBP-B, and GWBP-O on both platforms.

The first stage, *GPU kernelization with persistent residency* (GWBP-S), already provides clear gains: on Proteasome-A, it achieves 1.60 $\times$ speedup on V100 and 1.79 $\times$ on A100, while on Proteasome-B the gains are 1.17 $\times$ and 1.35 $\times$, respectively.

Adding *task batching and resource reuse* (GWBP-B) yields the largest improvement, raising the speedup to 3.99 $\times$ and 4.64 $\times$ on Proteasome-A, and to 2.95 $\times$ and 3.24 $\times$ on Proteasome-B. This confirms that, beyond kernel acceleration itself, reducing unnecessary work and system-level overhead is the dominant factor for end-to-end performance.

Finally, *efficient stream-based scheduling* (GWBP-O) further improves the runtime to 4.38 $\times$ and 4.78 $\times$ on Proteasome-A, and to 3.16 $\times$ and 3.33 $\times$ on Proteasome-B. The additional gain at this stage is smaller, because after the main bottlenecks have been removed, the remaining costs are less overlappable and increasingly bounded by fixed pipeline overheads.

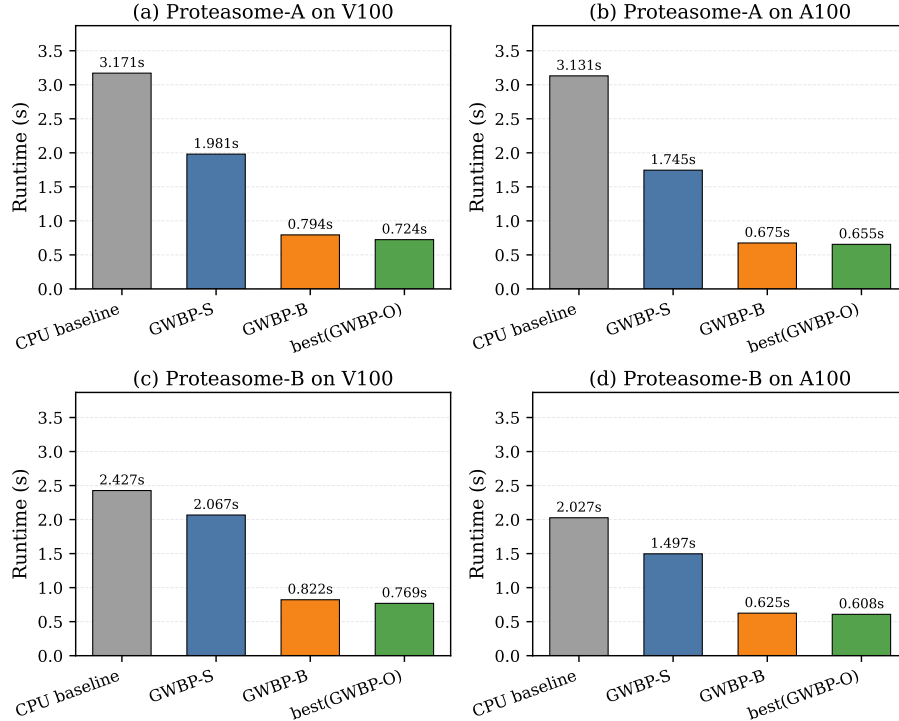


Fig. 5. Speedup progression across optimization stages on two platforms.

End-to-end Runtime and Speedup. Table 1 summarizes end-to-end runtime and speedup over the CPU baseline on both platforms and both datasets. Compared to the state-of-the-art multi-core CPU implementation, GWBP reduces end-to-end runtime from 3.131 s to 0.655 s on Proteasome-A with A100, achieving the best observed result of $4.78\times$ speedup. During single-GPU execution, NVIDIA-SMI sampling shows that the peak GPU memory consumption of GWBP-O is 2.63/1.81 GiB on Proteasome-A/B on V100 and 2.75/1.93 GiB on Proteasome-A/B on A100.

6 Conclusion

This paper presented GWBP, a highly optimized GPU implementation of WBP for cryo-ET reconstruction. GWBP accelerates the complete reconstruction pipeline rather than a single kernel: it kernelizes per-tilt operators and maintains persistent residency of intermediate data on the GPU, applies fragmentation-aware task batching and temporal buffer reuse to improve computational efficiency, and introduces a stream-oriented scheduling scheme to overlap communication with computation. Together, these optimizations reduce bottlenecks across weighting, 3D-CTF correction, and back-projection while improving overall throughput. Compared to the state-of-the-art multi-core CPU implementation, GWBP reduces end-to-end runtime from 3.131 s to 0.655 s, achieving up to a $4.78\times$ speedup.

Table 1. End-to-end runtime and speedup on two platforms.

Dataset	Method	Platform 1 (V100)		Platform 2 (A100)	
		Runtime (s)	Speedup	Runtime (s)	Speedup
Proteasome-A	CPU baseline	3.171	1.00×	3.131	1.00×
	GWBP-S	1.981	1.60×	1.745	1.79×
	GWBP-B	0.794	3.99×	0.675	4.64×
	best(GWBP-O)	0.724	4.38×	0.655	4.78×
Proteasome-B	CPU baseline	2.427	1.00×	2.027	1.00×
	GWBP-S	2.067	1.17×	1.497	1.35×
	GWBP-B	0.822	2.95×	0.625	3.24×
	best(GWBP-O)	0.769	3.16×	0.608	3.33×

Acknowledgments and Artifact Availability. This study was funded by the National Key Research and Development Program of China (No. 2023YFB3002404), the Guanghe Foundation (No. Ghfund202407010052). The artifact is available in the Zenodo repository [11]. The development version of GWBP is hosted on GitHub (<https://github.com/OptiYouth-Lab/GWBP>). We are committed to the long-term maintenance of the project.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Agulleiro, J.I., Fernandez, J.J.: Fast tomographic reconstruction on multicore computers. *Bioinformatics* **27**(4), 582–583 (2011). <https://doi.org/10.1093/bioinformatics/btq692>
2. Agulleiro, J.I., Fernandez, J.J.: Tomo3D 2.0—exploitation of advanced vector extensions (AVX) for 3d reconstruction. *Journal of Structural Biology* **189**(2), 147–152 (2015). <https://doi.org/10.1016/j.jsb.2014.11.009>
3. Burt, A., Toader, B., Warshamanage, R., von Kügelgen, A., Pyle, E., Zivanov, J., Kimanius, D., Bharat, T.A.M., Scheres, S.H.W.: An image processing pipeline for electron cryo-tomography in relion-5. *FEBS Open Bio* **14**(11), 1788–1804 (2024), <https://febs.onlinelibrary.wiley.com/doi/abs/10.1002/2211-5463.13873>
4. Cheng, A., Henderson, R., Mastronarde, D., Ludtke, S.J., Schoenmakers, R.H., Short, J., Marabini, R., Dallakyan, S., Agard, D., Winn, M.: Mrc2014: Extensions to the mrc format header for electron cryo-microscopy and tomography. *Journal of Structural Biology* **192**(2), 146–150 (2015). <https://doi.org/10.1016/j.jsb.2015.04.002>, recent Advances in Detector Technologies and Applications for Molecular TEM
5. Comet, M., et al.: Tomo Live: an on-the-fly reconstruction pipeline to judge data quality for cryo-electron tomography workflows. *Acta Crystallographica Section D* (2024), <https://journals.iucr.org/d/issues/2024/04/00/qv5004/>
6. Iudin, A., Korir, P.K., Somasundharam, S., Weyand, S., Cattavittello, C., Fonseca, N., Salih, O., Kleywegt, G., Patwardhan, A.: Empiar: the electron microscopy public image archive. *Nucleic Acids Research* **51**(D1), D1503–D1511 (11 2022). <https://doi.org/10.1093/nar/gkac1062>

7. Jiao, Y., et al.: TiltRec: an ultra-fast and open-source toolkit for cryo-electron tomographic reconstruction. *Bioinformatics* **41**(3) (2025), <https://academic.oup.com/bioinformatics/article/41/3/btaf068/8015615>
8. Kremer, J.R., Mastronarde, D.N., McIntosh, J.: Computer visualization of three-dimensional image data using imod. *Journal of Structural Biology* **116**(1), 71–76 (1996). <https://doi.org/10.1006/jsbi.1996.0013>
9. Kunz, M., Frangakis, A.S.: Three-dimensional ctf correction improves the resolution of electron tomograms. *Journal of Structural Biology* **197**(2), 114–122 (2017). <https://doi.org/10.1016/j.jsb.2016.06.016>, electron Tomography
10. Long, Y., Fessler, J.A., Balter, J.M.: 3d forward and back-projection for x-ray ct using separable footprints. *IEEE Transactions on Medical Imaging* **29**(11), 1839–1850 (2010). <https://doi.org/10.1109/TMI.2010.2050898>
11. Miao, H., Bian, H.: Artifact of the paper:gwbp: Accelerating weighted back-projection for image reconstruction via efficient gpu parallel optimization (Jun 2026). <https://doi.org/10.5281/zenodo.20506358>
12. Palenstijn, W.J., Bédorf, J., Sijbers, J., Batenburg, K.J.: A distributed ASTRA toolbox. *Advanced Structural and Chemical Imaging* **2**(1), 19 (2016). <https://doi.org/10.1186/s40679-016-0032-z>
13. Radermacher, M.: *Weighted Back-Projection Methods*, pp. 91–115. Springer US, Boston, MA (1992). https://doi.org/10.1007/978-1-4757-2163-8_5
14. Radermacher, M.: *Weighted Back-projection Methods*, pp. 245–273. Springer New York, New York, NY (2006). https://doi.org/10.1007/978-0-387-69008-7_9
15. Wang, Z., Zhang, Q., Mim, C.: Coming of age: Cryo-electron tomography as a versatile tool to generate high-resolution structures at cellular/biological interfaces. *International Journal of Molecular Sciences* **22**(12) (2021). <https://doi.org/10.3390/ijms22126177>
16. Zhang, K.: Gctf: Real-time ctf determination and correction. *Journal of Structural Biology* **193**(1), 1–12 (2016). <https://doi.org/10.1016/j.jsb.2015.11.003>
17. Zheng, K., Wu, K., Yao, Y., Bian, H., Huang, J., Wang, X.: Research on parallel weighted back-projection algorithm on multi-cpu. In: *Advanced Intelligent Computing Technology and Applications - 21st International Conference, ICIC 2025, Ningbo, China, July 26-29, 2025, Proceedings, Part XIV. Lecture Notes in Computer Science*, vol. 15855, pp. 193–204. Springer (2025). https://doi.org/10.1007/978-981-96-9908-7_16
18. Zheng, S., Wolff, G., Greenan, G., Chen, Z., Faas, F.G.A., Bárcena, M., Koster, A.J.: AreTomo: An integrated software package for automated marker-free, motion-corrected cryo-electron tomographic alignment and reconstruction. *Journal of Structural Biology: X* **6**, 100068 (2022). <https://doi.org/10.1016/j.yjsbx.2022.100068>
19. Zheng, Z., Mueller, K.: Cache-aware gpu memory scheduling scheme for ct back-projection. In: *IEEE Nuclear Science Symposium Medical Imaging Conference*. pp. 2248–2251 (2010). <https://doi.org/10.1109/NSSMIC.2010.5874183>
20. Zhou, L., Huang, C., Su, Y.: A fast back-projection algorithm based on cross correlation for gpr imaging. *IEEE Geoscience and Remote Sensing Letters* **9**(2), 228–232 (2012). <https://doi.org/10.1109/LGRS.2011.2165523>